
Travaux Dirigés Programmation

Découverte de la programmation, VS Code,
Python, C, HTML-CSS, PHP

Chapitres :

1. Découverte de la programmation et des outils de développement
2. Présentation de VS Code
3. Configuration de l'environnement de développement
4. Initiation à Python
5. Initiation au langage C
6. Initiation à HTML-CSS
7. Initiation à PHP

Année universitaire 2025–2026

1 Découverte de la programmation et des outils de développement

Définition – Programme et programmation

Un **programme** est une suite d'instructions écrites dans un langage compréhensible par une machine, destinée à résoudre un problème ou à réaliser une tâche. La **programmation** est l'activité de conception, d'écriture et de mise au point de ces programmes.

Exercice 1.1 – QCM : Généralités sur la programmation

Pour chaque question, choisissez la ou les bonnes réponses :

-) Un programme source est :
 -) Écrit en langage machine
 -) Écrit dans un langage de programmation
 -) Exécutable directement par le processeur
 -) Le résultat de la compilation
-) Le langage machine :
 -) Est composé d'instructions en binaire
 -) Est compréhensible par l'humain
 -) Est spécifique à chaque type de processeur
 -) Est le même que l'assembleur
-) Un compilateur :
 -) Traduit le code source en langage machine
 -) Exécute le programme ligne par ligne
 -) Produit un fichier exécutable
 -) Est nécessaire pour les langages interprétés
-) Un interpréteur :
 -) Traduit tout le code avant l'exécution
 -) Exécute les instructions une par une
 -) Nécessite une étape de compilation
 -) Est utilisé par Python

Exercice 1.2 – Langages compilés vs interprétés

Classez les langages suivants dans le tableau en indiquant s'ils sont **compilés**, **interprétés** ou **mixtes** (compilés + interprétés) :

Langage	Type (compilé / interprété / mixte)	
C		
Python		
Java		
JavaScript		
PHP		
C++		
HTML/CSS		

Exercice 1.3 – Vocabulaire de la programmation

Associez chaque terme à sa définition :

Terme	Définition
a. Algorithme	
b. Code source	
c. Bug	
d. IDE	
e. API	
f. Framework	

Définitions :

1. Ensemble d'outils logiciels structurants fournissant une architecture de base pour développer une application.
2. Suite finie et non ambiguë d'opérations permettant de résoudre un problème.
3. Interface permettant à des logiciels de communiquer entre eux.
4. Erreur dans un programme entraînant un dysfonctionnement.
5. Environnement de Développement Intégré (ensemble d'outils pour écrire, compiler et déboguer du code).
6. Fichier texte contenant les instructions écrites par le programmeur dans un langage de programmation.

Exercice 1.4 – Les étapes de la création d'un programme

Remettez dans l'ordre chronologique les étapes suivantes de la création d'un programme :

1. Compilation ou interprétation du code source
2. Analyse du problème et définition des besoins
3. Mise au point et tests (débogage)
4. Conception de l'algorithme
5. Écriture du code source
6. Maintenance et évolution

Exercice 1.5 – Types d'erreurs

Identifiez le type d'erreur pour chaque situation (erreur de syntaxe, erreur d'exécution, erreur logique) :

-) Oubli d'un point-virgule à la fin d'une instruction en C.
-) Division par zéro lors de l'exécution.
-) Un programme qui calcule la moyenne de 10 et 20 et affiche 300 au lieu de 15.
-) Nom de variable non déclaré utilisé dans le code.

-) Accès à un index de tableau qui n'existe pas.
-) Un programme qui trie une liste dans l'ordre décroissant alors qu'on voulait l'ordre croissant.

2 Présentation de VS Code

Qu'est-ce que VS Code ?

Visual Studio Code (VS Code) est un éditeur de code source gratuit, open source et multiplateforme, développé par Microsoft. Il supporte de nombreux langages de programmation et offre des fonctionnalités avancées grâce à son système d'extensions.

Exercice 2.1 – Installation de VS Code

1. Rendez-vous sur le site officiel : <https://code.visualstudio.com/>
2. Téléchargez la version correspondant à votre système d'exploitation (Windows, macOS ou Linux).
3. Procédez à l'installation avec les options par défaut.
4. Lancez VS Code et vérifiez que l'interface s'affiche correctement.

Exercice 2.2 – Identification des éléments de l'interface

Sur le schéma de l'interface de VS Code (fourni par le professeur ou observé sur votre écran), identifiez et nommez les éléments suivants :

-) La barre contenant les menus (Fichier, Édition, Affichage).
-) La zone principale où l'on écrit le code.
-) Le panneau affichant les fichiers et dossiers du projet.
-) La barre affichant des informations sur le fichier actif (langage, ligne, encodage).
-) Le panneau affichant les messages du terminal, des erreurs et des avertissements.
-) La barre latérale contenant les icônes d'accès rapide (Explorateur, Recherche, Extensions).

Exercice 2.3 – Raccourcis clavier essentiels

Complétez le tableau suivant avec les raccourcis clavier de VS Code (version Windows/Linux) :

Action	Raccourci
Nouveau fichier	
Ouvrir un fichier	
Sauvegarder	
Sauvegarder tout	
Annuler	
Refaire	
Rechercher dans le fichier	
Remplacer dans le fichier	
Ouvrir le terminal	
Commenter/Décommenter la ligne	

Exercice 2.4 – Extensions VS Code

Un étudiant souhaite développer en Python, C, HTML-CSS et PHP. Pour chaque langage, indiquez :

1. Le nom d'au moins **une extension** VS Code recommandée.
2. La fonction principale de cette extension.

Complétez le tableau :

Langage	Nom de l'extension	Fonction principale
Python		
C / C++		
HTML-CSS		
PHP		

Exercice 2.5 – Le terminal intégré

1. Ouvrez le terminal intégré de VS Code.
2. Tapez les commandes suivantes et observez le résultat :

```
pwd
ls
cd ..
cd ~
mkdir test_vscode
cd test_vscode
touch fichier_test.txt
ls
```

3. Expliquez brièvement ce que fait chaque commande.
4. Supprimez le dossier `test_vscode` et son contenu.

3 Configuration de l'environnement de développement

Principe

Avant de programmer, il faut configurer VS Code pour chaque langage : installer les extensions adaptées, configurer les tâches de compilation/exécution, et vérifier que le compilateur ou l'interpréteur est bien disponible sur la machine.

Exercice 3.1 – Vérification des outils installés

Ouvrez un terminal et vérifiez la présence (ou l'absence) des outils suivants sur votre machine :

```
python3 --version
gcc --version
php --version
node --version
git --version
```

Pour chaque commande, notez :

-) Si l'outil est installé ou non.
-) La version affichée le cas échéant.

Exercice 3.2 – Installation de Python

1. Si Python n'est pas installé, téléchargez-le depuis <https://www.python.org/downloads/>.
2. Lors de l'installation sous Windows, cochez impérativement l'option « Add Python to PATH ».
3. Vérifiez l'installation dans un terminal avec `python3 --version`.
4. Installez l'extension **Python** (Microsoft) dans VS Code.
5. Créez un fichier `test.py` contenant :

```
print("Hello, Python !")
```

6. Exécutez-le depuis le terminal intégré de VS Code.

Exercice 3.3 – Installation du compilateur GCC (C)

1. **Sous Windows** : Installez MinGW-w64 ou utilisez WSL (Windows Subsystem for Linux).
2. **Sous Linux** : Exécutez `sudo apt install gcc`.
3. **Sous macOS** : Installez Xcode Command Line Tools avec `xcode-select --install`.
4. Vérifiez l'installation avec `gcc --version`.
5. Installez l'extension **C/C++** (Microsoft) dans VS Code.
6. Créez un fichier `test.c` contenant :

```
#include <stdio.h>
int main() {
    printf("Hello, C !\n");
    return 0;
}
```

7. Compilez et exécutez :

```
gcc test.c -o test
./test
```

Exercice 3.4 – Installation de PHP

1. **Sous Windows** : Téléchargez PHP depuis <https://windows.php.net/download/> et ajoutez-le au PATH.
2. **Sous Linux** : `sudo apt install php php-cli`
3. **Sous macOS** : `brew install php`
4. Vérifiez avec `php --version`.
5. Installez l'extension **PHP Intelephense** dans VS Code.
6. Créez un fichier `test.php` contenant :

```
<?php
echo "Hello, PHP !";
?>
```

7. Exécutez : `php test.php`

Exercice 3.5 – Structure d'un projet

Pour un projet web complet utilisant Python, C, HTML-CSS et PHP, proposez une structure de dossiers cohérente. Dessinez l'arborescence sous la forme :

```
mon_projet/
|-- ...
|-- ...
```

Indiquez où vous placeriez :

-) Les fichiers Python
-) Les fichiers C
-) Les fichiers HTML et CSS
-) Les fichiers PHP
-) Les ressources (images, etc.)
-) La documentation

Exercice 3.6 – Fichier `settings.json` de VS Code

VS Code permet de personnaliser la configuration via le fichier `settings.json`. Proposez un fichier `settings.json` qui configure :

1. L'éditeur avec une taille de tabulation de 4 espaces.
2. Le formatage automatique à la sauvegarde.
3. L'interpréteur Python par défaut.
4. Le formateur par défaut pour les fichiers HTML/CSS.

4 Initiation et apprentissage de Python

Rappel – Python

Python est un langage interprété, typé dynamiquement, dont la syntaxe est proche du pseudo-code. Il est très utilisé en sciences, automatisation et développement web.

Exercice 4.1 – Premiers pas : print et input

Écrivez un programme Python qui :

1. Demande à l'utilisateur son nom avec `input()`.
2. Demande son âge.
3. Affiche : « Bonjour [nom], vous avez [âge] ans. »

Exercice 4.2 – Variables et types

Sans exécuter le code, déterminez le **type** de chaque variable après exécution :

```
a = 42
b = 3.14
c = "Bonjour"
d = True
e = a + b
f = a * 2
g = c + " monde"
h = str(a) + " ans"
```

Complétez le tableau :

Variable	Type	
a		
b		
c		
d		
e		
f		
g		
h		

Exercice 4.3 – Opérations et expressions

Écrivez un programme Python qui :

1. Demande deux nombres entiers a et b .
2. Calcule et affiche : leur somme, leur différence, leur produit, le quotient entier, le reste de la division, et la puissance a^b .
3. Affiche le type de chaque résultat.

Exercice 4.4 – Conditions

Écrivez un programme Python qui demande un entier n et affiche :

1. « Positif » si $n > 0$, « Négatif » si $n < 0$, « Zéro » si $n = 0$.

2. « Pair » ou « Impair ».
3. « Multiple de 3 » ou « Non multiple de 3 ».

Exercice 4.5 – Boucle `for`

Écrivez un programme Python qui :

1. Affiche les nombres de 1 à 10.
2. Affiche les nombres pairs de 2 à 20.
3. Calcule la somme des entiers de 1 à n (n saisi par l'utilisateur).
4. Affiche la table de multiplication d'un nombre n saisi par l'utilisateur.

Exercice 4.6 – Boucle `while`

Écrivez un programme Python qui :

1. Demande un mot de passe et redemande tant que ce n'est pas « python123 ».
2. Affiche les puissances de 2 jusqu'à dépasser 1000.
3. Devine un nombre : le programme choisit un nombre aléatoire entre 1 et 100, l'utilisateur devine, et le programme indique « Trop grand », « Trop petit » ou « Gagné! ».

Exercice 4.7 – Listes

Écrivez un programme Python qui :

1. Crée une liste contenant les entiers de 1 à 10.
2. Affiche le premier et le dernier élément.
3. Ajoute le nombre 11 à la fin.
4. Supprime le nombre 5.
5. Affiche la liste triée en ordre décroissant.
6. Calcule et affiche la somme et la moyenne des éléments.

Exercice 4.8 – Fonctions

Écrivez les fonctions Python suivantes :

1. `est_pair(n)` : retourne `True` si n est pair, `False` sinon.
2. `factorielle(n)` : retourne $n!$ (sans utiliser `math.factorial`).
3. `est_palindrome(mot)` : retourne `True` si le mot est un palindrome.
4. `moyenne(liste)` : retourne la moyenne d'une liste de nombres.

Exercice 4.9 – Chaînes de caractères

Écrivez un programme Python qui :

1. Demande une phrase à l'utilisateur.
2. Affiche le nombre de caractères.

3. Affiche la phrase en majuscules.
4. Affiche la phrase en minuscules.
5. Compte le nombre de voyelles.
6. Inverse la phrase.

5 Initiation et apprentissage du langage C

Rappel – Langage C

Le langage C est un langage compilé, typé statiquement, créé en 1972 par Dennis Ritchie. Il est à la base de nombreux systèmes d'exploitation (Unix/Linux) et reste fondamental en informatique.

Exercice 5.1 – Structure d'un programme C

Soit le programme C suivant :

```
#include <stdio.h>

int main() {
    printf("Bienvenue en C !\n");
    return 0;
}
```

1. Identifiez les trois parties principales de ce programme.
2. Expliquez le rôle de chaque ligne.
3. Que se passe-t-il si on retire le `#include <stdio.h>` ?

Exercice 5.2 – Variables et types en C

Déclarez en C les variables suivantes avec le type approprié :

Description	Déclaration C
Un entier valant 42	
Un nombre décimal valant 3.14	
Un caractère représentant la lettre 'A'	
Un entier long valant 1000000	
Un nombre décimal double précision valant 2.718	
Une chaîne « Bonjour »(tableau)	

Exercice 5.3 – printf et scanf

Écrivez un programme C qui :

1. Demande à l'utilisateur un entier n avec `scanf`.
2. Affiche : « Vous avez saisi : [n] » en utilisant `printf`.
3. Demande un nombre flottant x et l'affiche avec 2 décimales.

Exercice 5.4 – Opérateurs en C

Sans exécuter le code, donnez la valeur de chaque expression :

```
int a = 17, b = 5;
int c = a / b;
int d = a % b;
float e = (float)a / b;
int f = a++;
int g = ++b;
int h = (a > 10) ? 1 : 0;
```

Complétez le tableau :

Variable	Valeur
c	
d	
e	
f	
g	
h	

Exercice 5.5 – Conditions en C

Écrivez un programme C qui demande trois nombres et affiche le plus grand des trois.

Exercice 5.6 – Boucles en C

Écrivez un programme C qui :

1. Affiche les nombres de 1 à 10 avec une boucle `for`.
2. Calcule la somme des entiers de 1 à n avec une boucle `while` (n saisi par l'utilisateur).
3. Affiche le motif suivant pour $n = 5$:

```
*
**
***
****
*****
```

Exercice 5.7 – Tableaux en C

Écrivez un programme C qui :

1. Déclare un tableau de 5 entiers saisis par l'utilisateur.
2. Affiche le contenu du tableau.
3. Calcule et affiche la somme et la moyenne des éléments.
4. Trouve et affiche le maximum et le minimum.

Exercice 5.8 – Fonctions en C

Écrivez les fonctions C suivantes :

1. `int carre(int n)` : retourne le carré de n .

2. `int est_pair(int n)` : retourne 1 si n est pair, 0 sinon.
3. `int factorielle(int n)` : retourne $n!$.
4. `void swap(int *a, int *b)` : échange les valeurs de a et b .

Écrivez un programme principal qui teste chaque fonction.

6 Initiation et apprentissage de HTML-CSS

Rappel – HTML et CSS

HTML (HyperText Markup Language) définit la structure et le contenu d'une page web. **CSS** (Cascading Style Sheets) définit la présentation et la mise en forme. Un navigateur web interprète ces deux langages pour afficher des pages.

Exercice 6.1 – Structure d'un document HTML

Écrivez le code HTML5 minimal d'une page web contenant :

1. Le DOCTYPE.
2. La langue en français.
3. L'encodage UTF-8.
4. Un titre dans l'onglet : « Ma première page ».
5. Un titre principal `<h1>` : « Bienvenue! »
6. Un paragraphe avec du texte.

Exercice 6.2 – Balises HTML courantes

Sans exécuter le code, expliquez le rôle de chaque balise :

Balise	Rôle
<code><p></code>	
<code><a></code>	
<code></code>	
<code></code>	
<code></code>	
<code></code>	
<code><div></code>	
<code></code>	
<code><table></code>	

Exercice 6.3 – Listes et liens

Écrivez le code HTML pour :

1. Une liste non ordonnée de 4 langages de programmation.
2. Une liste ordonnée des étapes pour créer une page web.
3. Un lien vers le site de VS Code avec le texte « Télécharger VS Code ».
4. Une image avec un texte alternatif.

Exercice 6.4 – Tableaux HTML

Créez un tableau HTML avec :

1. Un en-tête de 3 colonnes : Langage, Type, Année de création.
2. 4 lignes de données (C, Python, Java, PHP).
3. Fusionnez les cellules de la première ligne sur les 3 colonnes pour le titre « Langages de programmation ».

Exercice 6.5 – Formulaires HTML

Créez un formulaire HTML avec :

1. Un champ texte pour le nom.
2. Un champ email.
3. Un champ mot de passe.
4. Une liste déroulante pour le langage préféré (C, Python, HTML-CSS, PHP).
5. Des boutons radio pour le niveau (Débutant, Intermédiaire, Avancé).
6. Une case à cocher « J'accepte les conditions ».
7. Un bouton de soumission.

Exercice 6.6 – Premiers pas en CSS

Écrivez le code CSS pour :

1. Mettre le texte de tous les paragraphes en bleu.
2. Centrer le titre `<h1>`.
3. Mettre un fond gris clair sur les `<div>`.
4. Ajouter une bordure de 2px solide noire aux tableaux.
5. Changer la police de toute la page en Arial.

Exercice 6.7 – Sélecteurs CSS

Sans exécuter le code, déterminez quels éléments seront affectés par chaque règle CSS :

```
/* Règle 1 */
h1 { color: red; }

/* Règle 2 */
.important { font-weight: bold; }

/* Règle 3 */
#principal { background: yellow; }

/* Règle 4 */
p em { color: green; }

/* Règle 5 */
div > p { margin: 10px; }
```

```
/* Règle 6 */  
a:hover { color: orange; }
```

Pour chaque règle, indiquez : le type de sélecteur (élément, classe, identifiant, descendant, enfant, pseudo-classe) et les éléments affectés.

Exercice 6.8 – Modèle de boîte CSS

1. Expliquez les quatre propriétés du modèle de boîte CSS : `content`, `padding`, `border`, `margin`.
2. Un `<div>` a les propriétés suivantes :

```
width: 200px;  
padding: 10px;  
border: 5px solid;  
margin: 15px;
```

Quelle est la largeur totale occupée par cet élément ?

Exercice 6.9 – Mise en page avec Flexbox

Créez une page HTML-CSS avec :

1. Un `<header>` contenant le titre du site.
2. Un `<nav>` avec un menu de 4 liens.
3. Un `<main>` contenant 3 `<article>` côte à côte.
4. Un `<footer>` avec des informations.
5. Utilisez Flexbox pour la mise en page.

7 Initiation et apprentissage de PHP

Rappel – PHP

PHP (PHP : Hypertext Preprocessor) est un langage de script côté serveur, principalement utilisé pour le développement web dynamique. Il s'intègre directement dans du code HTML et nécessite un serveur web (Apache/Nginx) pour fonctionner.

Exercice 7.1 – Premiers pas en PHP

Écrivez un script PHP qui :

1. Affiche « Bonjour, ceci est mon premier script PHP! ».
2. Affiche la date actuelle avec `date('d/m/Y')`.
3. Affiche les informations du serveur avec `phpinfo()`.

Testez votre script dans le terminal avec : `php script.php`

Exercice 7.2 – Variables et types en PHP

Sans exécuter le code, déterminez le type de chaque variable :

```
<?php
```

```
$nom = "Alice";
$age = 25;
$moyenne = 14.5;
$actif = true;
$notes = array(12, 15, 18);
$rien = null;
?>
```

Complétez le tableau :

Variable	Type PHP	
\$nom		
\$age		
\$moyenne		
\$actif		
\$notes		
\$rien		

Exercice 7.3 – Opérateurs en PHP

Sans exécuter le code, donnez le résultat de chaque expression :

```
<?php
$a = 10;
$b = 3;
echo $a + $b;
echo $a / $b;
echo $a % $b;
echo $a ** $b;
echo $a == 10 ? "oui" : "non";
echo $a === "10" ? "oui" : "non";
echo $a != $b ? "diff" : "egal";
echo isset($c) ? "def" : "undef";
?>
```

Exercice 7.4 – Conditions en PHP

Écrivez un script PHP qui :

- Définit une variable `$note` entre 0 et 20.
- Affiche la mention correspondante :
 - 0–9 : « Insuffisant »
 - 10–11 : « Passable »
 - 12–13 : « Assez bien »
 - 14–15 : « Bien »
 - 16–17 : « Très bien »
 - 18–20 : « Excellent »

Exercice 7.5 – Boucles en PHP

Écrivez un script PHP qui :

- Affiche les nombres de 1 à 10 avec une boucle `for`.
- Calcule la somme des entiers de 1 à n avec une boucle `while`.

3. Parcourt un tableau de prénoms avec `foreach` et affiche un message de bienvenue pour chacun.
4. Affiche la table de multiplication d'un nombre n .

Exercice 7.6 – Tableaux en PHP

Écrivez un script PHP qui :

1. Crée un tableau associatif contenant 5 élèves avec leurs notes.
2. Affiche tous les élèves et leurs notes.
3. Calcule et affiche la moyenne générale.
4. Trouve l'élève avec la meilleure note.
5. Trie le tableau par notes décroissantes.

Exercice 7.7 – Fonctions en PHP

Écrivez les fonctions PHP suivantes :

1. `est_pair($n)` : retourne `true` si `$n` est pair.
2. `factorielle($n)` : retourne `$n!`.
3. `est_palindrome($mot)` : retourne `true` si le mot est un palindrome.
4. `valider_email($email)` : retourne `true` si l'email est valide (utilisez `filter_var`).

Exercice 7.8 – Formulaires et PHP

Créez une page HTML avec un formulaire (méthode POST) contenant :

1. Un champ nom.
2. Un champ email.
3. Un champ message (textarea).
4. Un bouton d'envoi.

Puis écrivez le script PHP de traitement qui :

- a) Vérifie que tous les champs sont remplis.
- b) Valide le format de l'email.
- c) Affiche les données reçues de manière sécurisée (avec `htmlspecialchars`).

Exercice 7.9 – Sessions PHP

Écrivez un mini-système de connexion en PHP :

1. Une page de connexion (formulaire login/mot de passe).
2. Un script qui vérifie le couple login/mot de passe (en dur dans le code : admin / admin123).
3. Si les identifiants sont corrects, stocker le login dans `$_SESSION` et rediriger vers une page protégée.

4. La page protégée affiche « Bienvenue [login] » ou redirige vers la connexion si la session n'existe pas.
5. Une page de déconnexion qui détruit la session.